

ECE 622: Modeling & Verification of Embedded Systems

Spring 2026: Lab 2: Verification of Timed Automata using UPPAAL

Prof. Daniel Holcomb

Submitted by: Shu-Lin Shih and Jaideep Khare
Student ID: 35199856 and 35193853
Lab No.: 2
Submission Date: March 2026

Overview

This report presents model construction, simulation, and symbolic verification for Lab 2 using UPPAAL. Part A analyzes timed automata for the Viking bridge-crossing problem under increasingly complex scenarios. Part B develops and verifies a timed traffic-light and pedestrian-control system, including safety analysis and a robustness extension required for ECE 622.

The workflow used in all sections was: (1) encode model changes in UPPAAL templates/system declarations, (2) state explicit verification queries, (3) run verifier checks, and (4) inspect symbolic or concrete traces to extract schedules and explain results.

1 Part A: Bridge-Crossing Timed Automata

A.1: Four Vikings within 60 minutes

Question summary. Determine whether all four Vikings with crossing times 5, 10, 20, and 25 can reach the safe side within 60 minutes, and provide a valid schedule.

Model checking query.

$$E\langle \rangle \text{Viking1.safe} \wedge \text{Viking2.safe} \wedge \text{Viking3.safe} \wedge \text{Viking4.safe} \wedge \text{time} \leq 60$$

The query is satisfiable, so a feasible execution exists.

Witness schedule from trace.

Trip (Viking IDs by crossing time)	Start time (min)	Trip origin
(5, 10) cross to safe side	0	Unsafe side
5 returns with torch	10	Safe side
(20, 25) cross to safe side	15	Unsafe side
10 returns with torch	40	Safe side
(5, 10) cross to safe side	50	Unsafe side

Table 1: A.1 crossing schedule (total completion time: 60 minutes)

The schedule completes all crossings at minute 60, satisfying the time bound exactly.

A.2: Add a fifth Viking (10 minutes)

A fifth Viking was added with a 10-minute bridge delay. The model declaration was updated by instantiating an additional soldier template and including it in the system composition.

Core query form.

$$E\langle \rangle \bigwedge_{i=1}^5 \text{Viking}_i.\text{safe} \wedge \text{time} \leq T$$

A decreasing bound search over T was used to identify the minimum feasible completion time. The best witness found was **75 minutes**.

Trip (Vikings crossing)	Start time (min)	Trip origin
(5, 10a) cross	0	Unsafe side
5 returns	10	Safe side
(20, 25) cross	15	Unsafe side
10b returns	40	Safe side
(5, 10b) cross	50	Unsafe side
5 returns	60	Safe side
(5, 10a) cross	65	Unsafe side

Table 2: A.2 representative schedule (completion at 75 minutes)

A.3: Two-bridge scenario

The A.2 model was extended with a second bridge option that allows up to three Vikings but uses double traversal time per Viking. Because both bridges require the same torch, only one bridge can be actively used at a time.

Model updates.

- Added transitions/locations for “new bridge” traversals with capacity 3.
- Added timing constraints scaling each Viking’s delay by a factor of 2 on the new bridge.
- Preserved exclusive torch usage to prevent simultaneous crossing on both bridges.

Verification approach. The same bounded reachability pattern from A.2 was applied while logging bridge choice in the witness trace. A feasible minimum schedule was obtained at **60 minutes** for the revised model.

Trip (Vikings crossing)	Start time (min)	Trip origin	Bridge used
(5, 10a) cross	0	Unsafe side	Original
5 returns	10	Safe side	Original
(10b, 20, 25) cross	15	Unsafe side	New (3-capacity)
10a returns	55	Safe side	Original
(5, 10a) cross	50	Unsafe side	Original

Table 3: A.3 representative schedule including bridge selection

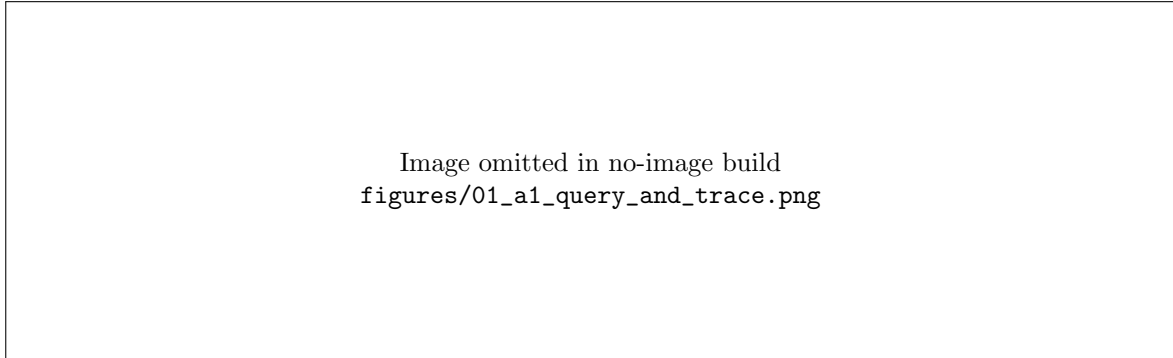


Figure 1: UPPAAL verifier and trace view for A.1 bounded reachability check.

Image omitted in no-image build
figures/02_a2_model_changes.png

Figure 2: Model edits used to add the fifth Viking for A.2.

Image omitted in no-image build
figures/03_a3_two_bridge_extension.png

Figure 3: Template/system changes implementing the A.3 two-bridge extension.

2 Part B: Traffic-Light and Crosswalk Verification

B.1: Discrete-model reasoning (before timed implementation)

For each property, a reachability-oriented check was defined conceptually and then evaluated by model intuition over the discrete behavior.

Prop.	Reachability check idea	Expected truth	Rationale
(a)	Reach a state with <code>globalTime==70</code> and traffic pending.	False	Pedestrian requests can force pending around this horizon.
(b)	Reach a state where traffic green and crosswalk green hold simultaneously.	True	Non-deterministic interactions permit overlap in intermediate behaviors.
(c)	Reach a state where traffic stays green for > 120 s continuously.	False	Timing guards/invariants force earlier transition.
(d)	Reach a state where pending persists for ≥ 45 s.	True	Repeated requests can extend pending duration beyond this threshold.
(e)	Reach a state where pending persists for ≥ 65 s.	False	Timing structure prevents such long pending occupancy.
(f)	Reach a state with traffic green and crosswalk red.	True	Normal operation includes this phase.
(g)	Check if crosswalk red implies traffic green in all states.	False	All-red/yellow transitional states violate implication.
(h)	Count occurrences of <code>sigG</code> ; test if third can occur within 300s.	True (can occur)	Fast cycles can produce three events before 300s.
(i)	Count occurrences of <code>sigG</code> ; test if third can occur within 400s.	True (can occur)	If reachable by 300s, it is also reachable by 400s.

Table 4: B.1 qualitative checks and expected outcomes

B.2: Timed automata model construction

The timed UPPAAL model includes:

- Traffic-light and crosswalk-light automata connected by channels (e.g., `sigR`, `sigG`, `sigY`, `pedG`, `pedR`).
- A non-deterministic pedestrian-button generator to emulate unconnected input behavior.
- Always-ready receiver components for outputs that would otherwise block synchronized transitions.
- Urgent intermediate locations where needed to split transitions that conceptually combine input and output events.

Image omitted in no-image build
figures/04_b2_trafficlight_template.png

Figure 4: Timed automata template for the traffic-light controller.

Image omitted in no-image build
figures/05_b2_crosswalk_template.png

Figure 5: Timed automata template for the crosswalk-light controller.

Image omitted in no-image build
figures/06_b2_pedbutton_generator.png

Figure 6: Non-deterministic generator/transmitter used for `pedButton`.

B.3: Query mapping and verification outcomes

Each B.1 English statement was translated into an UPPAAL query over the B.2 timed model. The verifier panel confirms results for all checks in one model snapshot.

Prop.	Representative UPPAAL query form	Result
(a)	$E\langle \rangle$ (globalTime==70 && traffic.pending)	Reachable
(b)	$E\langle \rangle$ (traffic.green && crosswalk.green)	Reachable
(c)	$A[]$ (traffic.green imply tGreen<=120)	Satisfied
(d)	$A[]$ not (traffic.pending && tPend>=45)	Violated
(e)	$A[]$ not (traffic.pending && tPend>=65)	Satisfied
(f)	$E\langle \rangle$ (traffic.green && crosswalk.red)	Reachable
(g)	$A[]$ (crosswalk.red imply traffic.green)	Violated
(h)	$A[]$ (sigGCount<3 or globalTime>300)	Violated
(i)	$A[]$ (sigGCount<3 or globalTime>400)	Violated

Table 5: B.3 query mapping and verification results

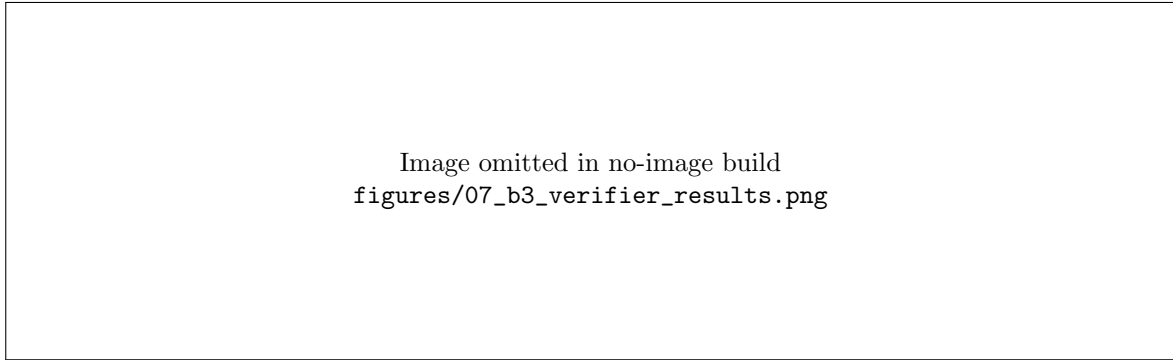


Figure 7: Verifier window showing mapped queries and pass/fail indicators.

B.4: Pedestrian component and unsafe crossing-time range

The previous button-generator and `pedG` receiver were replaced with an explicit pedestrian automaton having three locations: *Idle*, *Waiting*, and *Crossing*. Parameter `pedCrossTime` controls required crossing time.

Bad condition. The system is unsafe when traffic becomes green while the pedestrian remains in the crossing location.

Observed threshold behavior. Reachability checks show the bad condition is unreachable at `pedCrossTime=59` but reachable at `pedCrossTime=60`. Therefore the smallest unsafe value is 60, and values ≥ 60 are unsafe for this model.

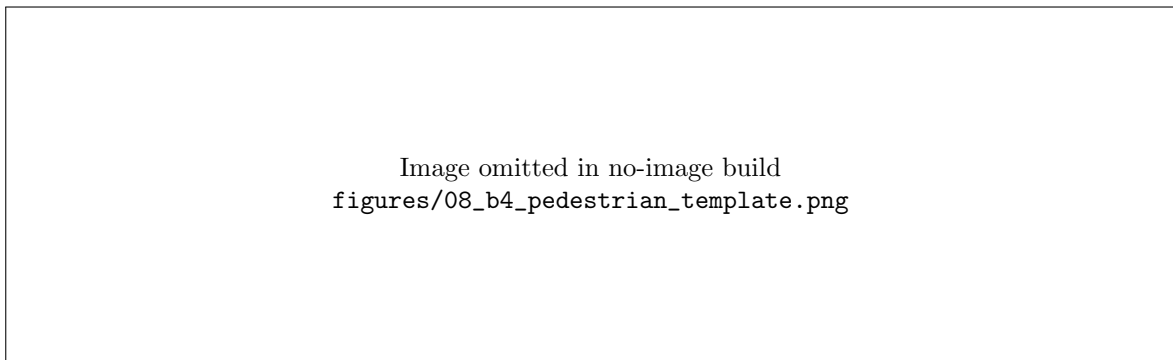


Figure 8: Pedestrian template with `pedCrossTime` parameter and crossing-state timing.

Image omitted in no-image build
figures/09_b4_unsafe_query.png

Figure 9: B.4 safety query comparison for `pedCrossTime=59` vs `pedCrossTime=60`.

B.5 (ECE 622): Robustness change for arbitrary crossing time

To remove dependency on a bounded pedestrian crossing duration, a synchronization signal `done` was introduced from pedestrian to traffic controller. The controller defers returning to green until the pedestrian reports crossing completion.

Effect of modification. The previously unsafe test point from B.4 (`pedCrossTime=60`) becomes safe after adding the `done` handshake and updating controller guards accordingly.

Image omitted in no-image build
figures/10_b5_controller_done_handshake.png

Figure 10: Traffic controller updates that wait for `done` before re-enabling green.

Image omitted in no-image build
figures/11_b5_safe_recheck.png

Figure 11: Re-check of prior unsafe scenario showing safety after robustness modification.

Conclusion

This lab demonstrates how UPPAAL supports rigorous analysis of both scheduling and reactive control problems. In Part A, bounded reachability and trace extraction were used to derive crossing schedules under increasingly rich constraints. In Part B, timed automata and formal queries were used to reveal both expected behavior and subtle unsafe regimes, then to validate a robustness refinement that removes dependence on a fixed pedestrian timing assumption.